

Project Description

A **Containerized Item Exchange Application with Full DevOps Lifecycle** is a web-based item exchange platform that allows users to post items they no longer need and either offer them for free or exchange them with other users (no selling).

The application consists of a full web system, including both **frontend and backend** components along with the database. From a DevOps perspective, the project will focus on **implementing the full DevOps lifecycle** — including containerization, Orchestration , continuous integration and deployment and infrastructure automation.

By the end of the project, the platform will demonstrate a **production-grade DevOps workflow** deployed on **AWS**, showcasing automated build, test and deployment processes in a scalable and reliable environment.

Objective

1. Deliver a functional full-stack item-exchange website (frontend + backend) deployed on AWS.
2. Implement a complete DevOps lifecycle: version control → CI → automated tests → image build → automated deployment .
3. Automate infrastructure provisioning and server configuration using IaC (Terraform).
4. The application services using a lightweight, production-grade K3s Kubernetes cluster for high availability, scaling, and self-healing capabilities.
5. Demonstrate the workflow with an automated demo.

Tools & Technologies

- **Containerization & Orchestration:** Docker, Kubernetes (K3S)
- **Image Registry:** Amazon ECR
- **CI/CD:** Jenkins
- **Cloud:** AWS (EC2, RDS)
- **IaC:** Terraform

Milestones & Deadlines

Week 1 — Containerization & orchestration

- Dockerize the app(front + back).
- Set up the K3s and deploy initial Kubernetes manifests.

Week 2 —CI/CD Pipeline Setup

- Configure Jenkins for automated build and deployment

Week 3 —Set up AWS Environment & IaC

- Create IAM roles , Implement Infrastructure-as-Code

Week 4 — Deployment Automation

- Integrate CI to provision & deploy to AWS (push image to ECR, run container).

Week 5— Final Testing, Documentation & Demo

- End-to-end tests, performance checks, finalize README and slides, record demo.

Key Performance Indicators (KPIs)

1. Infrastructure & Automation

- **laC completeness:** 100% of infrastructure resources are provisioned via Terraform.

2. Pipeline Efficiency & Performance

- **Pipeline time target:** Full CI/CD run (tests + build + push + deploy) completes in **≤ 10 minutes** for typical changes.
- **Deployment downtime:** Zero manual downtime during deployment (target: **< 1 minute** or rolling update with no user-visible downtime).

3. Deployment & Cloud Management

- **Successful automated deploys:** **≥ 95%** of CI-triggered deployments succeed without human rollback.
- **Image registry:** All production images are tagged and present in ECR

4. Orchestration & Cluster Health

- **Pod Readiness:** All application pods (Frontend, Backend) reach the Ready state within 30 seconds of deployment.
- **Ingress Functionality:** Traefik Ingress successfully routes both root (/) and API traffic (/api) to the correct services.

Stakeholder Analysis

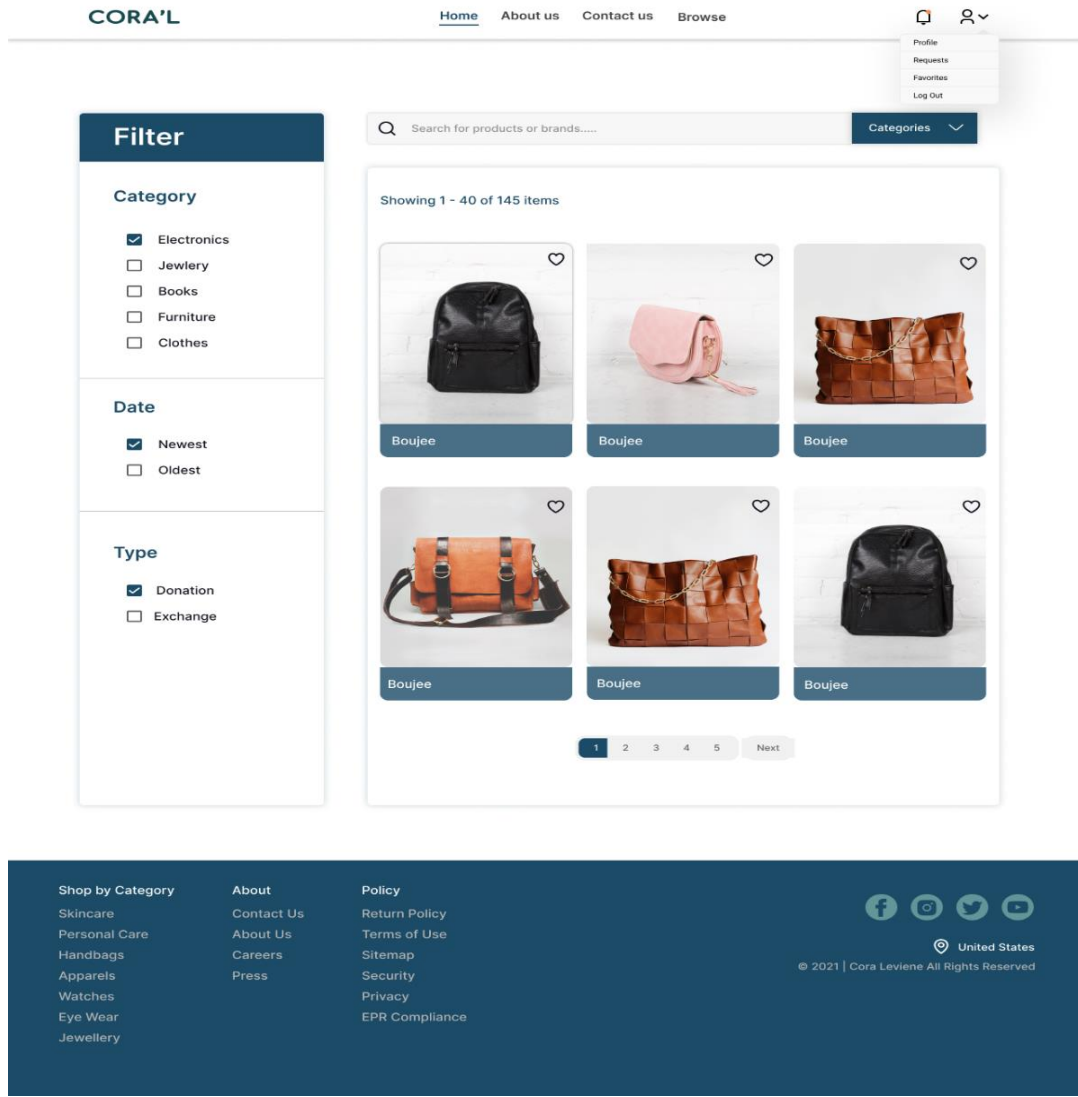
This project involves several stakeholders who contribute to its success at different stages of the DevOps lifecycle.

Stakeholder	Role
Team Leader: Sama Mostafa Fazaa	- Design & implement CI/CD (Jenkins). - Automate build and deploy pipeline to AWS.
Team member 1: Ahmed Hamdy Mahmoud Elshewemy	- Write Terraform templates. - Keep infrastructure versioned and reusable.
Team member 2: Omar Said Omar	- Create AWS account and monitor AWS costs. - Configure VPCs and security groups
Team member 3: Osama Mamdouh Mohamed	- Create Docker files for the item-exchange app - Build and push images to Amazon ECR
Team member 4: Mahmoud Gomaa Abouelfetouh	- Deploy and manage the K3s cluster on EC2. - Configure Traefik Ingress, Services, and Deployments.
Team member 5: Mayar Khaled Mostafa	- Configure the AWS RDS PostgreSQL instance. - Manage database connectivity and private subnet configuration.

Instructor / Supervisor: Muhammed Gamal	Evaluates and guides the project
End Users	Use the item exchange platform
Cloud Provider: AWS	Hosts and supports deployment

UI/UX Design

Home page:



Product page:

CORA'L

[Home](#)
[About us](#)
[Contact us](#)

Clothes

lorem ipsum dolor takayak malay holy

Request Item

Add To Wishlist

Contact Seller

4/10/2022

<

>

Shop by Category

Skincare

Personal Care

Handbags

Apparels

Watches

Eye Wear

Jewellery

About

Contact Us

About Us

Careers

Press

Policy

Return Policy

Terms of Use

Sitemap

Security

Privacy

EPR Compliance

United States

© 2021 | Cora Leviene All Rights Reserved

Sign up page:

Home

WELCOME TO AWESOMENESS

Please create your free account to proceed

Already signed up? [LOGIN](#)

Create free account

USERNAME

your unique username

MAIL ADDRESS

me@mymail.com

PASSWORD

enter password

Create my account

OR

Sign up with Google

By signing up, you agree with our communications and usage terms.

User profile page:

CORA'L

[Home](#)[About us](#)[Contact us](#)[Browse](#)





Sara Dahman

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book.

Add New Item



Lorem Ipsum is simply dummy text of the printing and typesetting industry

4/10/2022 | ART

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book.



Lorem Ipsum is simply dummy text of the printing and typesetting industry

4/10/2022 | ART

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book.

EditDelete

Shop by Category

Skincare

Personal Care

Handbags

Apparels

Watches

Eye Wear

Jewellery

About

Contact Us

About Us

Careers

Press

Policy

Return Policy

Terms of Use

Sitemap

Security

Privacy

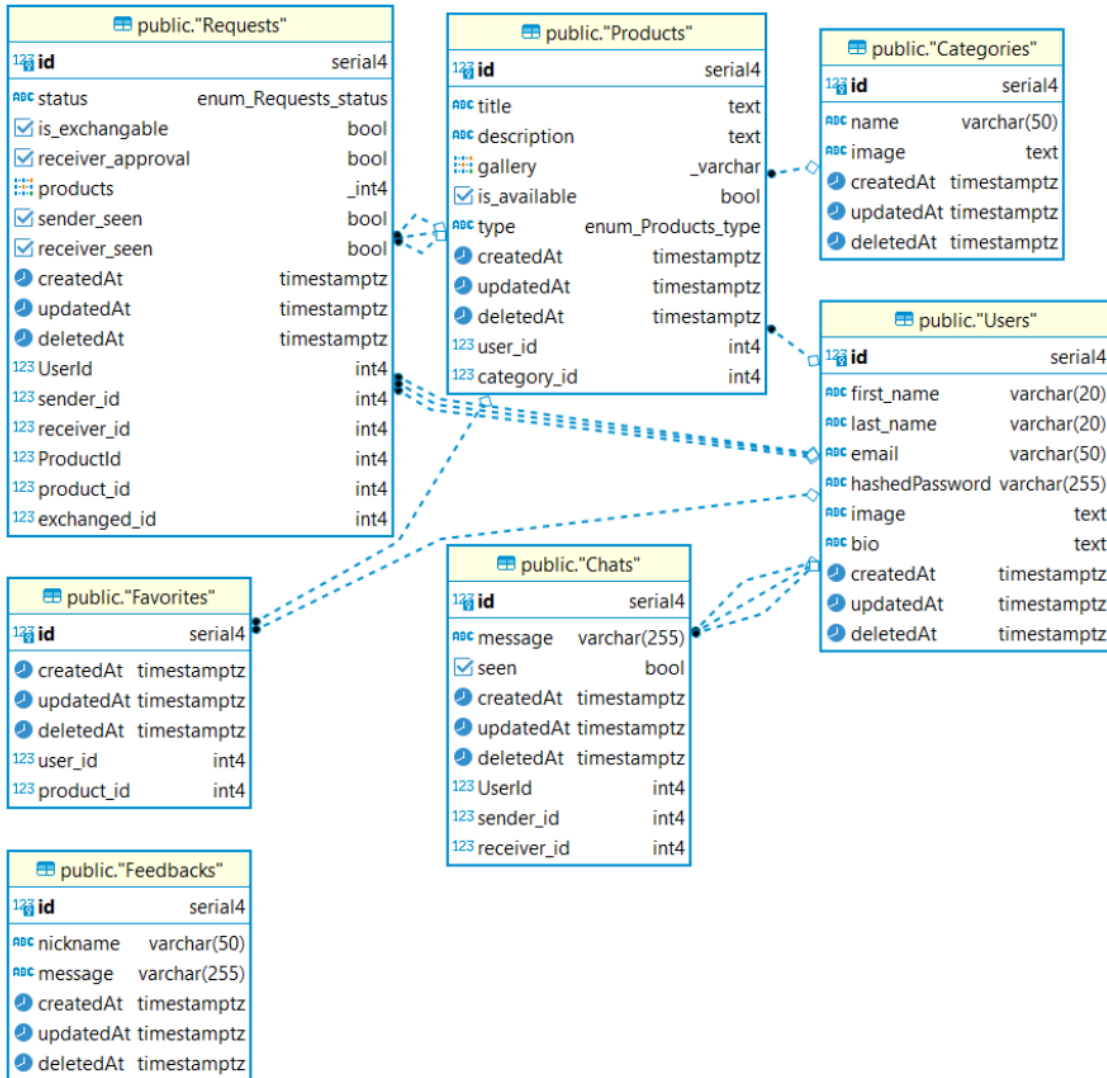
EPR Compliance



 United States

© 2021 | Cora Leviene All Rights Reserved

Database Design

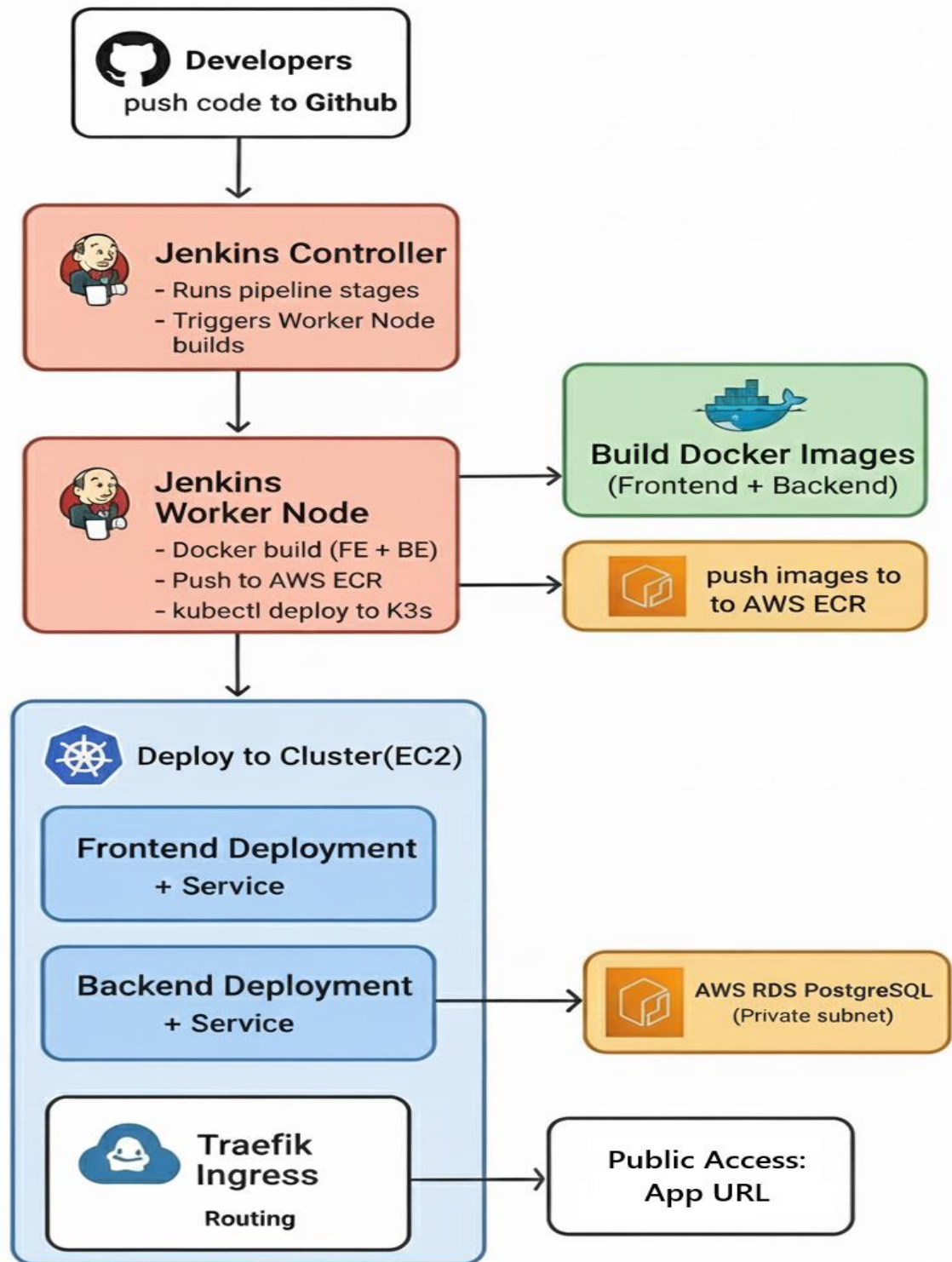


Key Features

This project is built upon several best-in-class DevOps practices and technologies, offering the following key features:

- **Fully Automated CI/CD Pipeline:** The entire software delivery lifecycle, from a code push to GitHub to application deployment on Kubernetes, is automated using **Jenkins**, ensuring rapid, repeatable, and error-free releases.
- **Distributed Build Architecture:** **Jenkins Controller** manages job orchestration, while a dedicated **Jenkins Worker Node** handles resource-intensive tasks like Docker image builds and large file transfers, ensuring the controller remains stable and responsive.
- **Lightweight Kubernetes Orchestration (K3s):** A resource-efficient **K3s** cluster is used to manage the deployment, scaling, and self-healing of the application's microservices, providing a production-ready environment on AWS EC2.
- **Secure & Managed Data Storage:** **AWS RDS PostgreSQL** is provisioned in a **Private Subnet**, completely inaccessible from the public internet. Access is strictly limited via Security Groups to only the **Backend Pods** within the K3s cluster.
- **Intelligent Traffic Routing (Traefik Ingress):** A single public entry point manages traffic distribution. **Traefik** handles path-based routing: directing general requests (/) to the Frontend service and API calls (/api) to the Backend service.

Architecture Design



Implementation Phases

Phase 1: Containerization & Registry:

The goal was to package the application for consistency and portability across all environments.

- Dockerfile.frontend: Implements a multi-stage build pattern. The first stage compiles the React application, and the second, final stage copies the compiled assets into a lightweight Nginx base image, which serves the static files.
- Dockerfile.backend: Sets up the Node.js runtime environment, installs dependencies, and runs the Express/Socket.io server.
- Output: Two distinct, versioned Docker images ready for registry storage.

AWS ECR(Elastic Container Registry) Setup:

- Two AWS ECR repositories were created: jack-frontend and jack-backend.
- Integration: The Jenkins Worker Node was configured with IAM credentials to authenticate with ECR, allowing automated tagging and pushing of new images upon every pipeline run.

Phase 2: AWS Infrastructure Setup:

Three EC2 instances were created:

1. K3s Cluster Node -> Runs K3s Kubernetes and hosts the entire application.
2. Jenkins Controller -> Orchestrates CI/CD pipeline execution.
3. Jenkins Worker Node -> Runs Docker builds and kubectl deployments.

AWS RDS (Relational database service):

- An AWS RDS PostgreSQL instance was provisioned for the database.
- Networking: It was placed within a Private Subnet to prevent external access.
- Security: A dedicated Security Group was created to explicitly allow inbound traffic on port 5432 (PostgreSQL) *only* from the K3s Cluster Node's internal IP range, ensuring the Backend pod is the only application able to communicate with the database.
- Connectivity: The backend connects using the DATABASE_URL environment variable.

Phase 3: K3s Deployment & Ingress:

The application services were defined and deployed to the cluster.

- A lightweight, production-grade K3s cluster was installed on the dedicated EC2 instance.
- Components Deployed:
 - Deployments: Manage the desired state of the Frontend and Backend pods.
 - Services (ClusterIP): Provide stable internal endpoints for the deployments.
 - ConfigMap/Secrets: Used to inject necessary configuration (e.g., API keys, database connection string).
 - Traefik Ingress: Configured to expose the services to the public, routing traffic based on path.
- Application Access: The website became publicly available at the EC2 instance's IP: <http://16.170.173.4>.

Phase 4: Jenkins CI/CD Pipeline:

The pipeline automates the entire process, linking code changes to production updates.

- Architecture:
 - Jenkins Controller → Manages jobs
 - Jenkins Worker → Builds Docker, pushes images, deploys to K3s.
- Pipeline Stages (Implemented in Groovy):
 - stage('Checkout'): Fetches the latest code from the GitHub repository.
 - stage('Login to ECR'): Authenticates the Worker Node with AWS ECR Public.
 - stage('Build Frontend') / stage('Build Backend'): Executes the respective docker build commands.
 - stage('Push Images'): Tags the newly built images and runs docker push to ECR.
 - stage('Deploy to K3s'): Executes kubectl apply -f [manifests] to roll out the new image versions in the K3s cluster.

Project Results & KPIs Achievement

This project successfully delivered a fully functional, containerized, and CI/CD-enabled application, meeting all critical Key Performance Indicators (KPIs) mentioned above.

Live Application(on production): [App Url](#)

Future Enhancements

- Monitoring and Observability:
 - Integrate the Prometheus and Grafana stack into the K3s cluster to collect and visualize key metrics (CPU/Memory usage, network latency, application error rates).
- Security & Compliance:
 - Implement ExternalDNS and deploy cert-manager to provision and automatically renew free SSL certificates (using Let's Encrypt), securing the application with HTTPS.

Conclusion

The **Containerized Item Exchange Application** project successfully demonstrated a modern DevOps tools and cloud-native practices. By combining the automation power of **Jenkins**, the container management of **K3s**, and the robust cloud services of **AWS**, we established a resilient, scalable, and fully automated deployment pipeline capable of supporting rapid development and continuous delivery.